

Pricing API — Project Summary

Spring Boot REST API for managing product prices, price configurations and sales data.

1. Project Structure

Layer	Class	Responsibility
Model	Product	Product entity (id, name, price)
Model	Sale	Sale entity (product, date, qty, revenue)
Model	PriceConfiguration	Price config entity (product, price, currency, dates)
DTO	PriceConfigurationRequest	JSON request body for price config
DTO	DailySalesSummary	Response for daily sales query
DTO	OptimumPriceResponse	Response for optimum price query
Repository	ProductRepository	CRUD for products
Repository	SaleRepository	Sales queries — daily & optimum price
Repository	PriceConfigurationRepository	Overlap query for price configs
Service	PricingService	Product & sales business logic
Service	PriceConfigurationService	Overlap-aware price config upsert
Controller	PricingController	Product & sales REST endpoints
Controller	PriceConfigurationController	Price config REST endpoint

2. REST Endpoints

Method	URL	Description
POST	/api/products	Create a product
GET	/api/products	List all products
PUT	/api/products/{id}/price?price=99.9	Update product price
POST	/api/sales?productId=1&quantity=5 &saleDate=2025-01-15	Record a sale
GET	/api/sales/daily?date=2025-01-15	Daily sales summary
GET	/api/sales/optimum-price	Product with highest revenue
PUT	/api/price-configurations	Add or update price config (JSON body)

3. PriceConfiguration — JSON Request Body

Send this JSON body to **PUT /api/price-configurations**:

```
{
  "productId" : 2,
  "price"      : 49.99,
  "currency"   : "GBP",
  "startDate"  : "2025-03-01",
  "endDate"    : "2025-03-10"
```

```
}
```

4. Overlap Handling Logic

When a new price configuration is submitted, any existing records for the same product that overlap the new date range are automatically adjusted. No request is ever rejected.

Case 1 — Existing fully covered by new

Action: **DELETE existing**

```
Existing:  |--Mar03---Mar07--|
New:       |--Mar01-----Mar10--|
Result:    |--Mar01-----Mar10--|
```

Case 2 — New fully inside existing

Action: **SPLIT existing into two**

```
Existing:  |--Mar01-----Mar10--|
New:       |--Mar04---Mar07--|
Result:    |--Mar01-Mar03--| |--Mar04---Mar07--| |--Mar08-Mar10--|
```

Case 3 — Existing overlaps on the right

Action: **TRIM existing start**

```
Existing:  |--Mar05-----Mar15--|
New:       |--Mar01---Mar08--|
Result:    |--Mar01---Mar08--| |--Mar09-Mar15--|
```

Case 4 — Existing overlaps on the left

Action: **TRIM existing end**

```
Existing:  |--Mar01---Mar08--|
New:       |--Mar05-----Mar15--|
Result:    |--Mar01-Mar04--| |--Mar05-----Mar15--|
```

5. Transaction & Atomicity

The **addOrUpdate** method is annotated with:

```
@Transactional(isolation = Isolation.SERIALIZABLE, rollbackFor = Exception.class)
```

- **SERIALIZABLE isolation** — prevents concurrent transactions from inserting overlapping records between the read and write phases.
- **rollbackFor = Exception.class** — any failure (delete or insert) rolls back the entire operation, leaving the database in its original state.
- **toDelete list** — all existing records to remove are batched and executed first via `deleteAll()`.
- **toAdd list** — all trimmed/split records plus the new config are batched and executed via `saveAll()`. The new config is always last in the list.

6. Build & Run

```
./gradlew clean bootRun
```

H2 Console: <http://localhost:8080/h2-console>

Field	Value
JDBC URL	jdbc:h2:mem:pricingdb
User Name	sa
Password	(leave blank)